

Sélection et filtrage de données

Objectifs d'apprentissage

- Comprendre comment sélectionner et filtrer des données

Vous avez été brièvement initiés à la sélection et au filtrage de données dans les modules précédents, mais il s'agit d'un concept tellement important que nous souhaitons lui consacrer un module entier.

Sélection par indices

Vous avez déjà appris que certains éléments d'un vecteur peuvent être sélectionnés en spécifiant un indice :

```
x <- 55:65

# Appeler x sans sélectionner
x
## [1] 55 56 57 58 59 60 61 62 63 64 65
```

```
# Maintenant, appeler uniquement le troisième élément de x
x[3]
## [1] 57
```

Rappel : les crochets indiquent que vous ne souhaitez pas extraire tous les éléments d'un vecteur ; vous souhaitez seulement certains d'entre eux. Je veux `x` , *mais pas tout*.

Vous pouvez également extraire un sous-ensemble d'un objet en appelant plusieurs indices :

```
# Appelons maintenant le troisième, le quatrième et le cinquième élément de x

x[c(3,4,5)]
## [1] 57 58 59
```

```
# Autre façon de faire la même chose :

x[3:5]
## [1] 57 58 59
```

Sélection de sous-ensembles avec des booléens

Vous pouvez également sélectionner des sous-ensembles d'objets à l'aide de booléens. Ce sera de loin votre méthode de filtrage de données la plus courante.

Rappelons que les données booléennes (ou logiques) ont deux valeurs possibles : `TRUE` ou `FALSE` . Par exemple :

```
# Stocker l'âge de Joe
joes_age <- 35

# Définir le seuil de vieillesse
old_age <- 36

# Vérifier si Joe est âgé
joes_age >= old_age
## [1] FALSE
```

N'oubliez pas non plus que vous pouvez calculer si une condition est `TRUE` ou `FALSE` sur plusieurs éléments d'un vecteur. Par exemple :

```
# Créer un vecteur d'âges multiples
ages <- c(10, 20, 30, 40, 50, 60)

# Définir le seuil de vieillesse
old_age <- 36

# Déterminer quels âges sont considérés comme vieux
ages >= old_age
## [1] FALSE FALSE FALSE TRUE TRUE TRUE
```

Les vecteurs booléens sont très utiles pour la sélection de sous-ensembles. « Sous-ensemble » signifie ne conserver que les éléments d'un vecteur pour lesquels une condition est vraie.

```
x <- 55:59

# Appel de x sans sous-ensemble
x
## [1] 55 56 57 58 59
```

```
# Sous-ensemble du deuxième, troisième et quatrième élément
x[c(FALSE, TRUE, TRUE, TRUE, FALSE)]
## [1] 56 57 58
```

Cette commande a renvoyé les éléments pour lesquels le vecteur de sous-ensemble était `TRUE` .

Ceci est équivalent à...

```
x[2:4]
## [1] 56 57 58
```

Vous pouvez également obtenir le même résultat en utilisant un test logique, car les tests logiques renvoient des valeurs booléennes :

```
# Élaborez votre test logique : demandez quelles valeurs de x sont dans le vecteur 56:58
x %in% c(56,57,58)
## [1] FALSE TRUE TRUE TRUE FALSE

# Maintenant, insérez ce test dans les crochets de sous-ensemble

x[ x %in% c(56,57,58) ]
## [1] 56 57 58
```

Cette méthode devient très utile lorsque vous travaillez avec de grands ensembles de données, comme celui-ci :

```
# Créer un grand ensemble de données de nombres aléatoires

y <- sample(1:1000,size=100)
length(y)
## [1] 100
```

```
range(y)
## [1] 2 956
```

Avec un ensemble de données comme celui-ci, vous pouvez utiliser un filtre booléen pour déterminer combien de valeurs sont supérieures à, par exemple, 90.

Commencez par développer votre test logique, qui vous indiquera si chaque valeur du vecteur est supérieure à 90 :

```
# Développer votre test logique,

y > 90
## [1] TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE
## [13] TRUE TRUE TRUE TRUE FALSE TRUE TRUE FALSE TRUE TRUE TRUE TRUE
## [25] TRUE FALSE TRUE TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
## [37] TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE FALSE TRUE TRUE TRUE TRUE
## [49] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [73] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [85] TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [97] TRUE TRUE TRUE FALSE
```

Maintenant, pour obtenir les valeurs correspondantes à Pour chaque TRUE de cette liste, insérez votre test logique entre les crochets de sélection.

```
y[y > 90]
## [1] 860 125 433 661 825 120 932 517 925 91 956 274 780 319 385 238 256 249 356
## [20] 776 186 608 124 712 100 886 399 300 347 779 266 280 432 879 507 292 250 293
## [39] 632 209 230 899 201 349 437 861 821 471 408 326 704 351 444 392 313 407 645
## [58] 312 278 491 418 686 354 754 294 410 413 402 388 431 343 474 807 556 373 98
## [77] 384 561 139 463 207 404 151 804 543 769 511 220
```

Voici une autre façon de procéder :

```
# Enregistrez le résultat de votre test logique dans un nouveau vecteur
verdicts <- y > 90

# Utilisez ce vecteur pour extraire un sous-ensemble de y

y[verdicts]
## [1] 860 125 433 661 825 120 932 517 925 91 956 274 780 319 385 238 256 249 356
## [20] 776 186 608 124 712 100 886 399 300 347 779 266 280 432 879 507 292 250 293
## [39] 632 209 230 899 201 349 437 861 821 471 408 326 704 351 444 392 313 407 645
## [58] 312 278 491 418 686 354 754 294 410 413 402 388 431 343 474 807 556 373 98
## [77] 384 561 139 463 207 404 151 804 543 769 511 220
```

Vous pouvez également utiliser des tests logiques doubles. Par exemple, que se passe-t-il si vous souhaitez obtenir tous les éléments compris entre 70 et 90 ?

```
verdicts <- y > 70 & y < 90
y[verdicts]
## integer(0)
```

Exercice de révision

1. Créez un vecteur nommé `nummies` contenant tous les nombres de 1 à 100.
2. Créez un autre vecteur nommé `little_nummies` contenant tous les nombres inférieurs ou égaux à 30.
3. Créez un vecteur booléen nommé `these_are_big` indiquant si chaque élément de `nummies` est supérieur ou égal à 70.
4. Utilisez `these_are_big` pour créer un sous-ensemble de `nummies` dans un vecteur nommé `big_nummies`.
5. Créez un nouveau vecteur nommé `these_are_not_that_big` indiquant si chaque élément de `nummies` est supérieur à 30 et inférieur à 70. Il faut utiliser le symbole `&`.
6. Créez un nouveau vecteur nommé `meh_nummies` contenant tous les nombres `nummies` supérieurs à 30 et inférieurs à 70.
7. Combien y a-t-il de nombres supérieurs à 30 et inférieurs à 70 ?
8. Quelle est la somme de tous les nombres contenus dans `meh_nummies` ?